Attention and transformers

Attention and self attention
Transformers architecture with Encoder and Encoder-Decoder
Applications for text classification
Language models basics and translation
Examples with ***pytorch***

Recent developments on sequence processing

Previously, we saw two types of methods that could help in processing sequential data:

Embeddings – helped to create more compact representations of tokens (e.g. words) and with semantics (latent spaces with meaning); They launched the idea of representing concepts with compact number vectors by learning (with gradient descent) and showed that it can work well

Recurrent networks – allow to work on sequential data directly; create mechanisms of memory (in LSTMs of short and long term) to process the sequences... BUT ... can have difficulties with with long sequences and with the correct understanding of context in natural language.

*I was born and raised in **Portugal**. Later, I lived in France, Italy, Germany and the UK, and travelled to many parts of the world. I learned different cultures and habits, I met interesting people and even learned to speak some words in other languages. Still, the only language I speak fluently is **Portuguese**.*

Timeline

A bit of an exaggeration... but things change fast in these fields... yet LSTMs dominated the field until at least 2018...

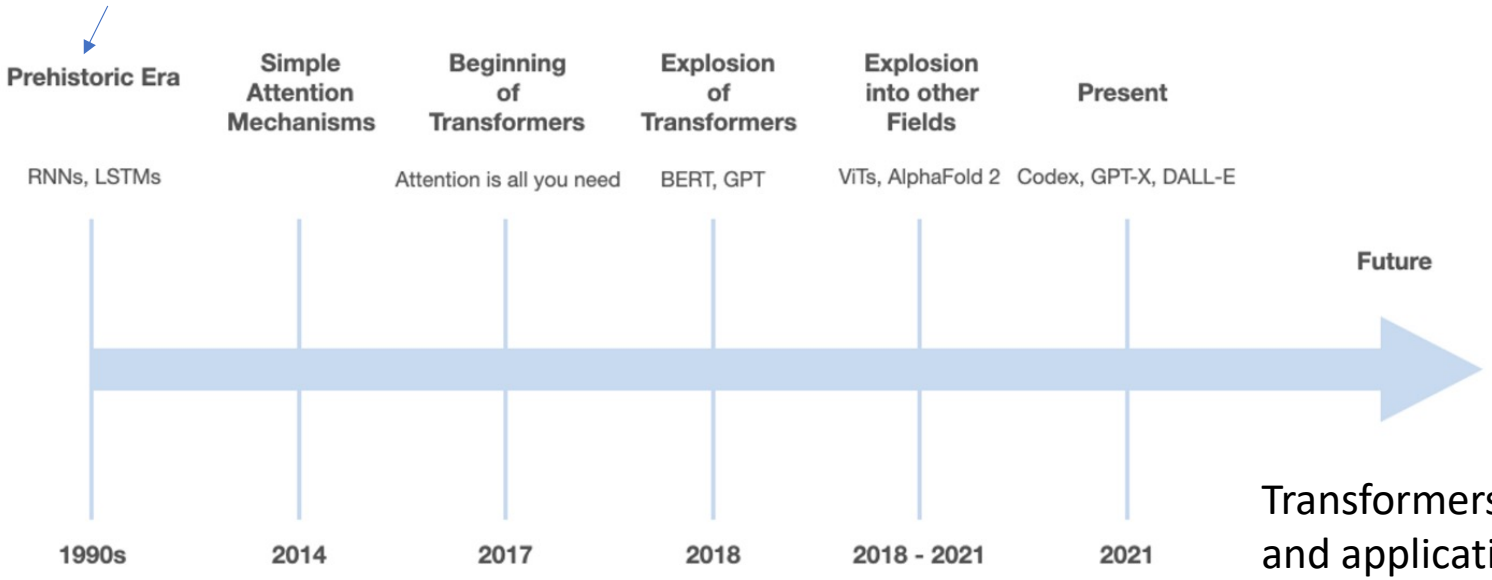


Figure taken from the "Transformers United" course taught at Stanford
Available at https://web.stanford.edu/class/cs25/

Transformers are used in many areas and applications today including vision, reinforcement learning, few-zero shot learning.
They are the basis of the latest LLMs and prompt systems such as ChatGPT and DALLE-2

Simple attention mechanisms

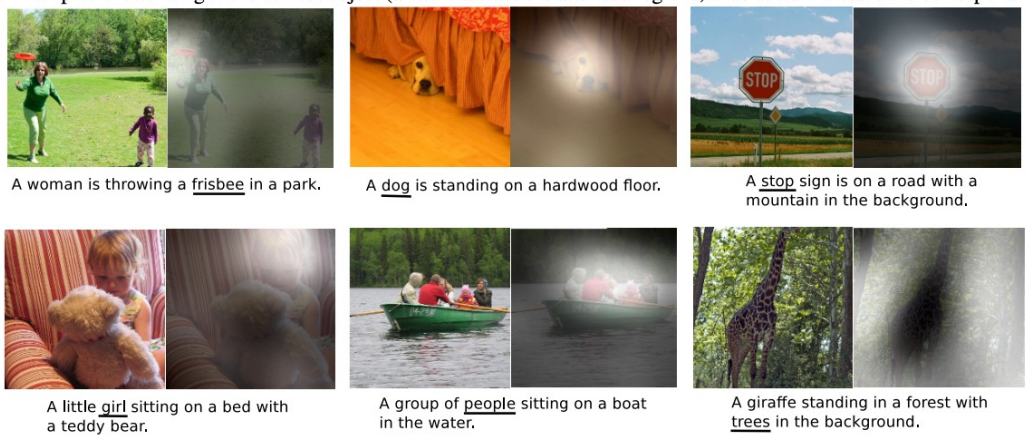
Attention mechanism was proposed for images as a way to give different weights to different parts of the images, with two variants:

- ***Soft attention***: weights between [0,1] throughout the image
  - Differentiable models
  - But they involve heavy computations because they work on the whole image
- ***Hard attention***: weights only in some parts of the images
  - Non-differentiable models which made training difficult
  - But fewer calculations in inference

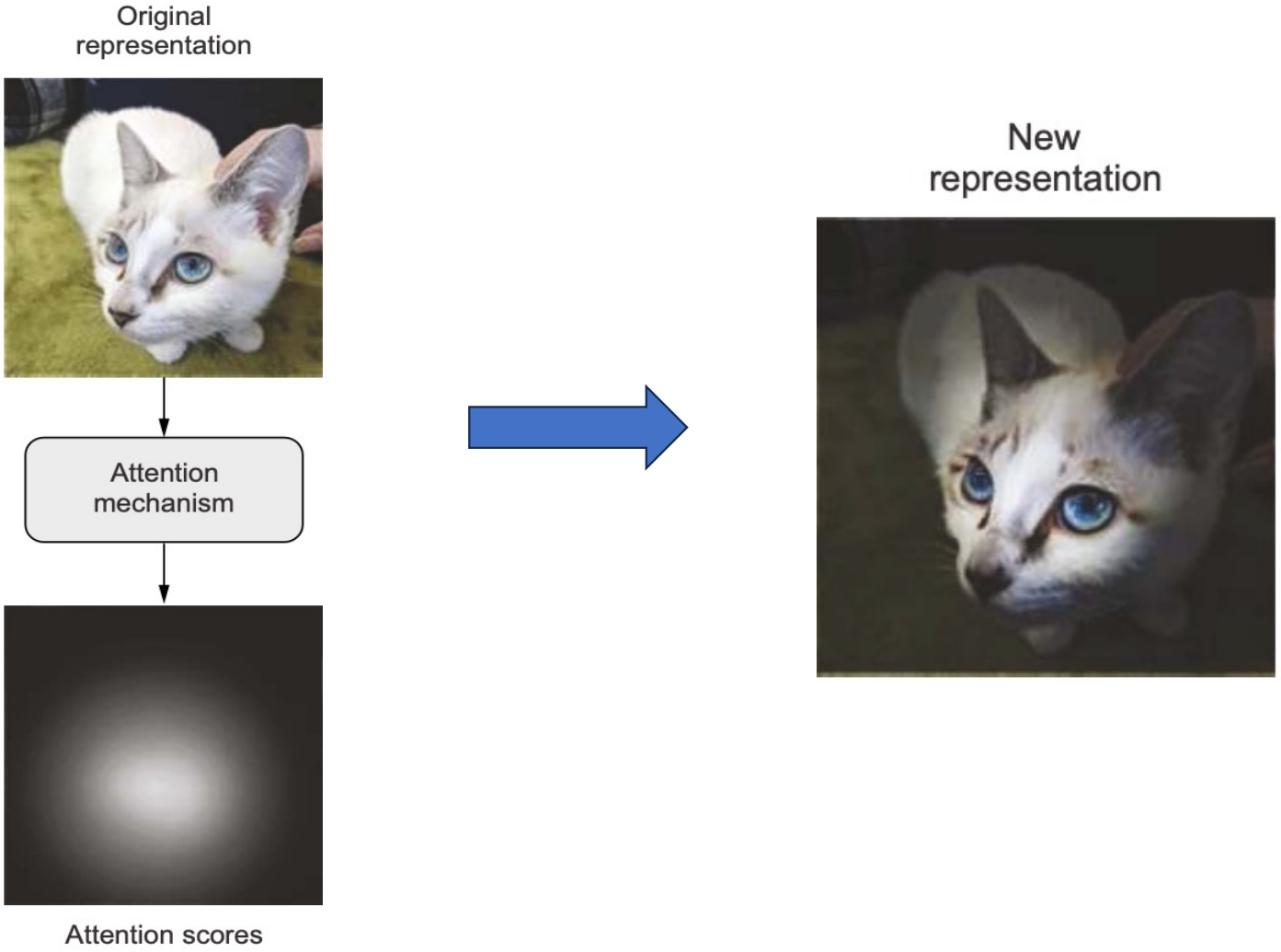
https://lilianweng.github.io/posts/2018-06-24-attention/

Paper: http://proceedings.mlr.press/v37/xuc15.pdf

Figure 4. Examples of attending to the correct object (*white* indicates the attended regions, *underlines* indicated the corresponding word)



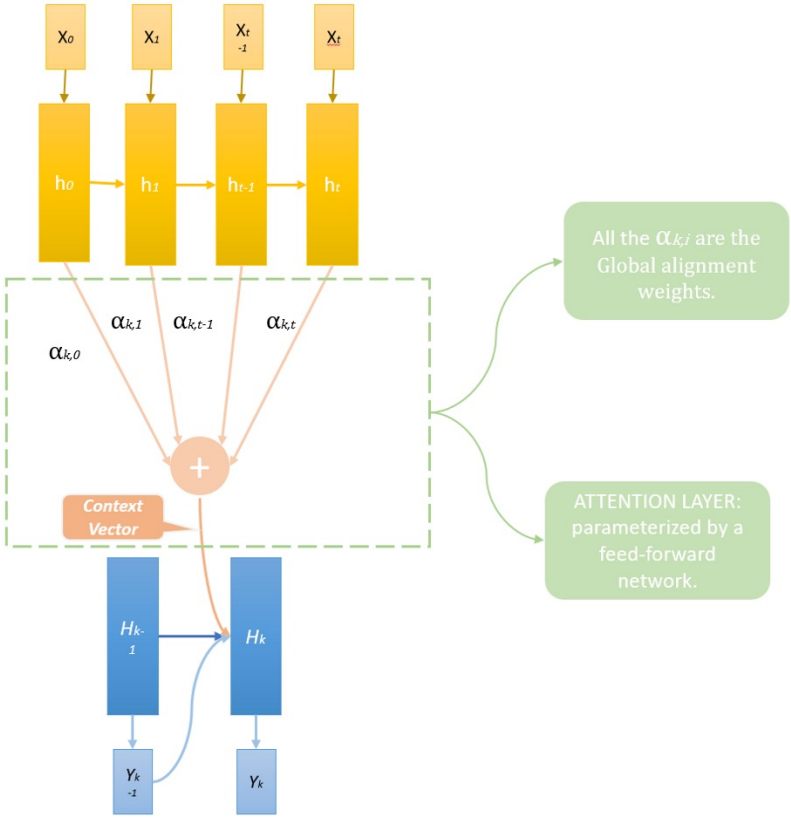
Attention mechanism: illustration



F. Chollet

Attention in texts: Neural machine translation

- Attention mechanism proposed for RNNs that implement machine translation as a way to overcome problems in decoders in representing the entire initial sequence processed in a single context vector
- Idea is to use the outputs of all intermediate steps in the decoder, with specific weights (learned in the training process)
- Allows to **identify more relevant parts of the sequence** when processing the input for the result, can be applied locally or globally



Paper - 2015: https://nlp.stanford.edu/pubs/emnlp15_attn.pdf
More details:
https://towardsdatascience.com/intuitive-understanding-of-attention-mechanism-in-deep-learning-6c9482aecf4f
https://towardsdatascience.com/attaining-attention-in-deep-learning-a712f93bdb1e

Attention in texts

In texts, **attention mechanisms** are in a way similar to **TF-IDF normalization** which assigns importance scores to tokens based on how much information different tokens are likely to carry. Important tokens get boosted while irrelevant tokens get faded out.

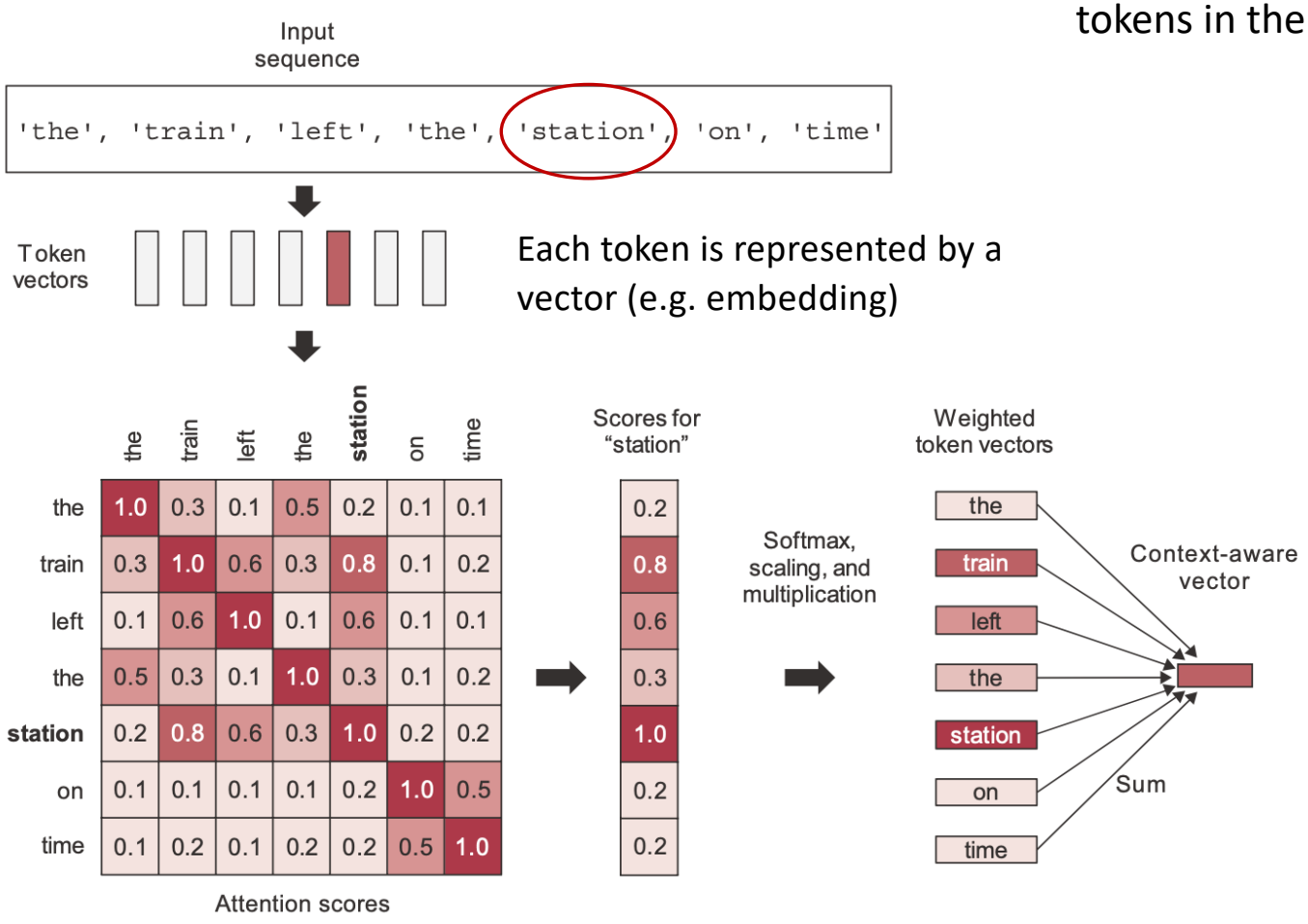
Another analogy is to look at latent representations (e.g. **embeddings**) which have a fixed numerical representation for each word. But, these do not depend on context

We can look at **attention mechanisms** in texts as **representations** that should depend on the **context** of the work in the text

*Peter and Mary went on a **date** but did not get along.*
*The project's final **date** is May 27th.*
*Peter did the like the **date** he had for desert since it was too sweet.*

Self attention in texts

Purpose: produce context-aware token representations, by modulating the representation of a token by using the representations of related tokens in the sequence.



Step 1

Step 2

Step 1:
compute **attention scores –** similarity between the vector for the target word (in this case "station") and every other word in the sentence – use dot product between their representations

Step 2:
compute the **sum** of all word vectors in the sentence, weighted by attention scores, after **normalization and scaling**; resulting vector is the new representation for the target word, incorporating context

This process is repeated for all the words in the sentence

F. Chollet

Self attention in texts

- The process we saw before can be summarized in:

```
outputs = sum (inputs * pairwise_scores(inputs, inputs))
```

- The generalized version of self-attention

```
outputs = sum (values * pairwise_scores(query, keys))
```

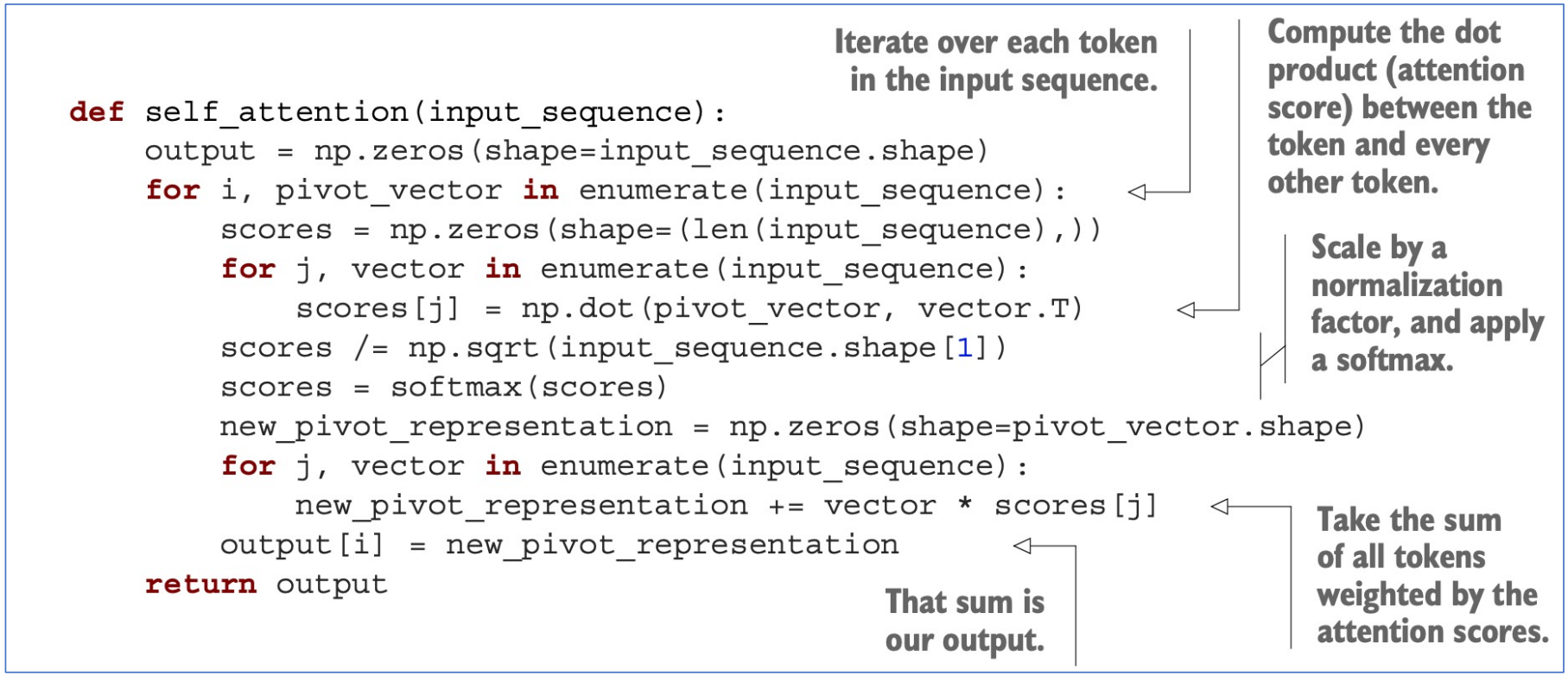
For each element in the ***query***, compute how much the element is related to every ***key***, and use these scores to weight a sum of ***values***

In practice, the keys and the values are often the same sequence.

F. Chollet

Self attention in texts - code

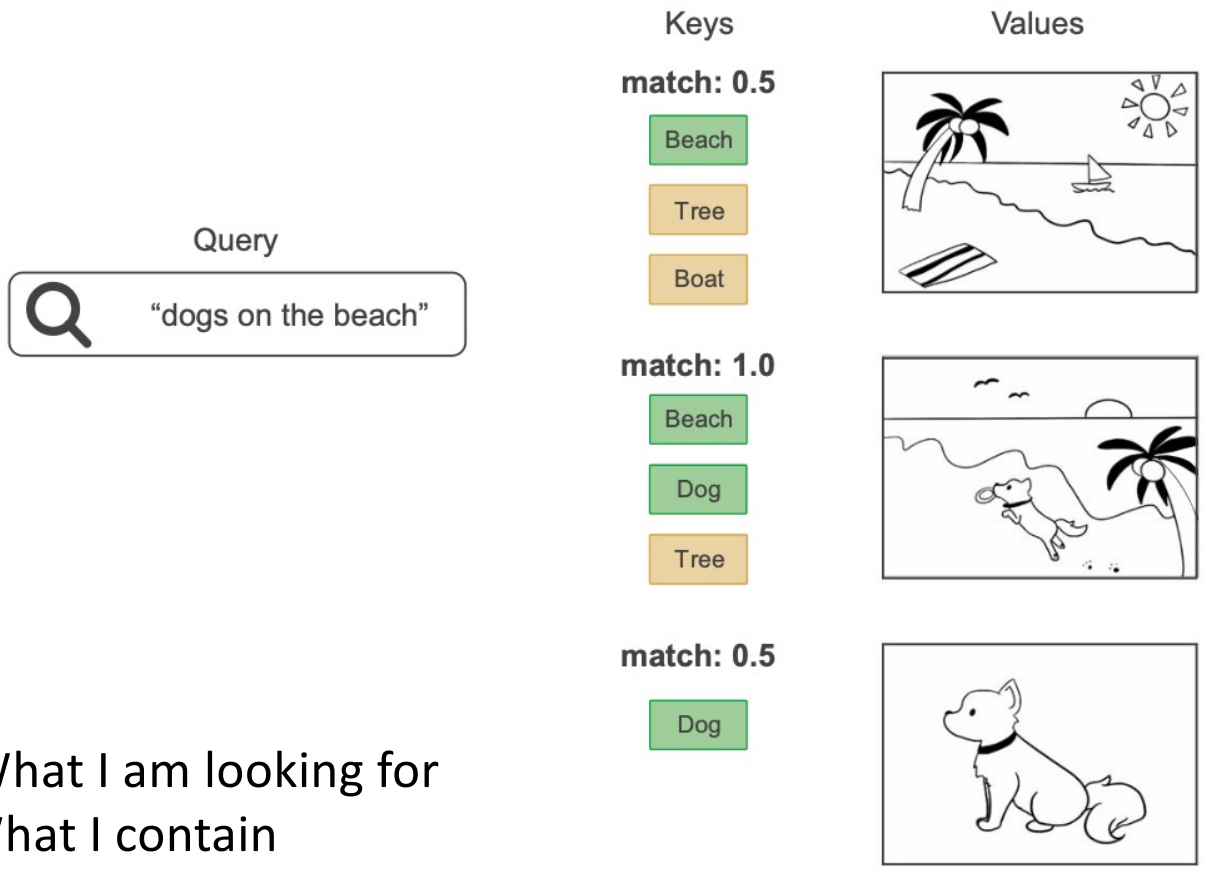
NumPy implementation




```
def self_attention(input_sequence):
    output = np.zeros(shape=input_sequence.shape)
    for i, pivot_vector in enumerate(input_sequence):
        scores = np.zeros(shape=(len(input_sequence),))
        for j, vector in enumerate(input_sequence):
            scores[j] = np.dot(pivot_vector, vector.T)
        scores /= np.sqrt(input_sequence.shape[1])
        scores = softmax(scores)
        new_pivot_representation = np.zeros(shape=pivot_vector.shape)
        for j, vector in enumerate(input_sequence):
            new_pivot_representation += vector * scores[j]
        output[i] = new_pivot_representation
    return output
```

F. Chollet

Self attention in texts: analogy to search



Analogy to image retrieving from a database indexed by keywords (our keys).

The query is compared to these keys and the matching scores are used to rank images ("values")

Q – What I am looking for
K – What I contain
V – Information I pass

F. Chollet

Self-attention in transformers

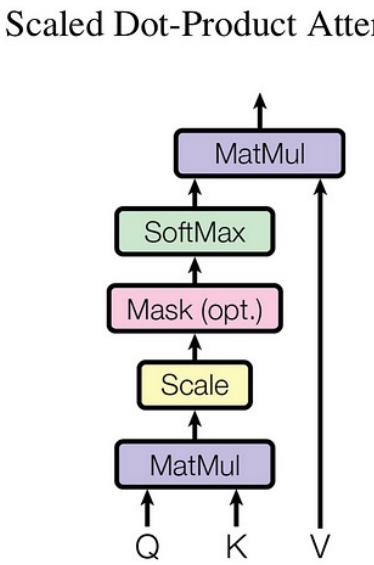
In transformers, the mechanism of self-attention works with three vectors, which are all obtained from the input multiplying by 3 **different matrices**:

- Q – query – vector representation of each word in the sequence
- K – keys – vector representations of all words in the sequence
- V – values – identical to K, but it will be multiplied by the weights of each word

Multiplication of Q by K^T aims to identify similarities and calculate attention scores

Scores scaled and normalized to produce weights

Multiply the weights by V: applies the weights to the vector V to recover the values



$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Self attention in torch

```
class SelfAttention(nn.Module):

    def __init__(self, d_model):
        super().__init__()
        self.q = nn.Linear(d_model, d_model)
        self.k = nn.Linear(d_model, d_model)
        self.v = nn.Linear(d_model, d_model)

    def forward(self, x):
        Q = self.q(x)
        K = self.k(x)
        V = self.v(x)

        scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(Q.size(-1))
        weights = torch.softmax(scores, dim=-1)
        output = torch.matmul(weights, V)

        return output
```

Simple implementation of one head of a self attention layer

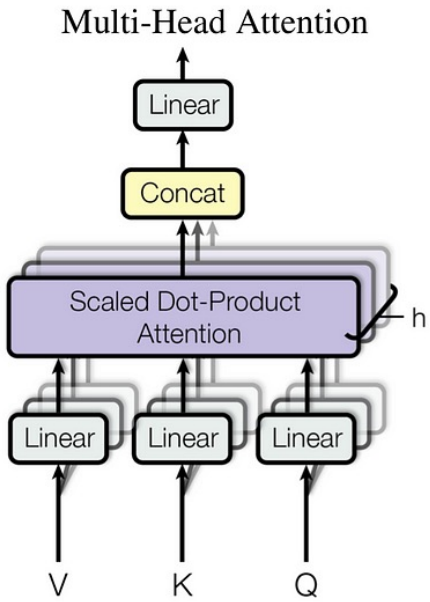
Python code:
minimal-transformer.ipynb

Multi-head attention

Several layers of attention are placed in parallel in a module multi-head attention; here, K,Q, V are multiplied by learned weights (linear projections)

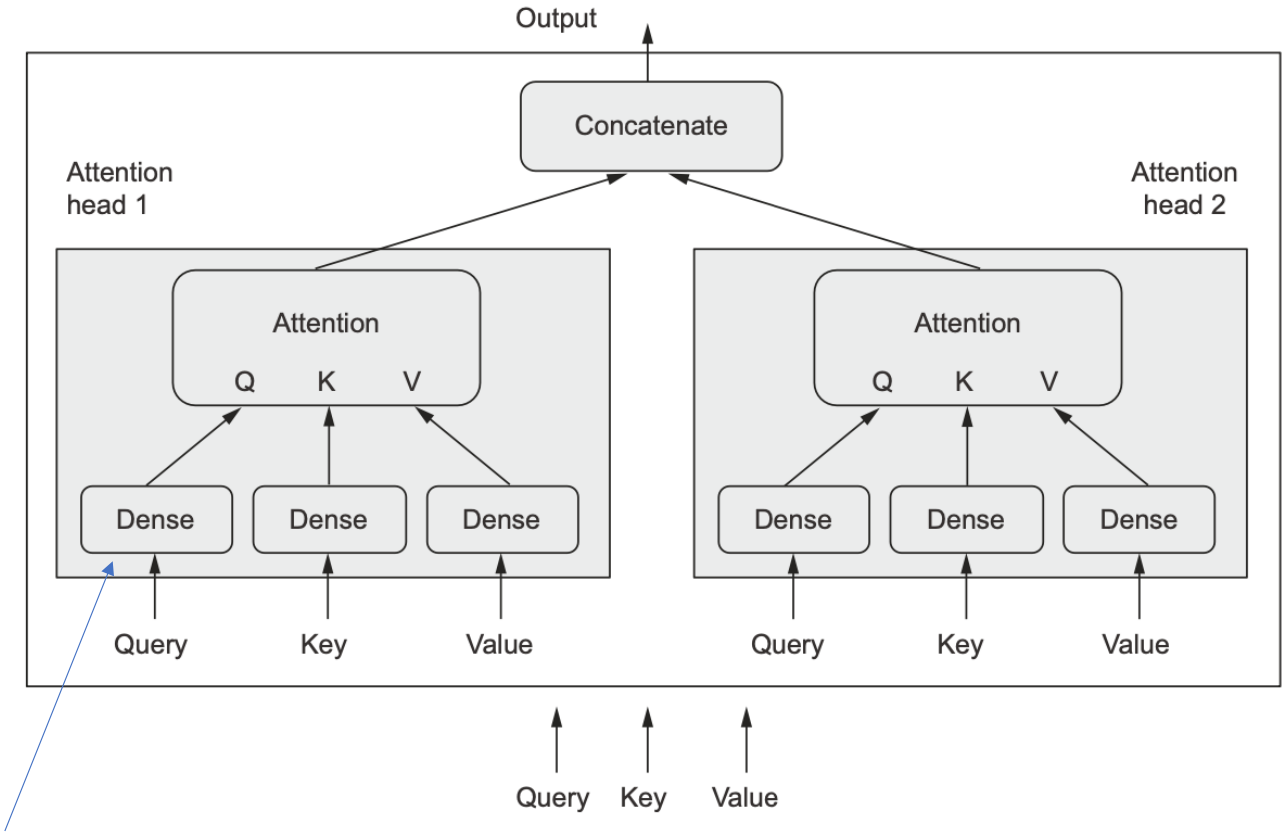
Independent heads helps the layer learn different groups of features for each token, where features within one group are correlated with each other but are mostly independent from features in a different group.

Examples of possible features: syntactic relations, long-distance dependencies, positional relationships, semantic relations



Detailed description of the processes:
https://e2eml.school/transformers.html

Multi-head attention



The output of the self-attention layer gets factored into a set of independent sub-spaces, learned separately

The Q, K, V are sent through three independent sets of dense weights, resulting in three separate vectors.

Each vector is processed via attention, and the three outputs are concatenated back together into a single output sequence.

Dense layers enable the attention layer to actually learn something, as opposed to being a purely stateless transformation

F. Chollet

Multi-head attention in torch

```
class MultiHeadAttention (nn.Module):

    def __init__(self, d_model, num_heads):
        super().__init__()
        self.heads = nn.ModuleList(
            [SelfAttention(d_model) for _ in range(num_heads)] )
        self.linear = nn.Linear(d_model * num_heads, d_model)

    def forward(self, x):
        head_outputs = [head(x) for head in self.heads]
        concat = torch.cat(head_outputs, dim=-1)
        output = self.linear(concat)

        return output
```

Concatenates the output of the different heads and applies a Dense layer

Python code:
minimal-transformer.ipynb

Transformers

- Transformers are models that use the attention mechanism explained above (***self attention***)
- Implement Seq2Seq models (convert sequences into new sequences, using an Encoder and a Decoder), **without using recurrent networks** but only distinct feed forward layers
- Encoder – encodes text into latent representations (numerical)
- Decoder – Decodes from latent space to text

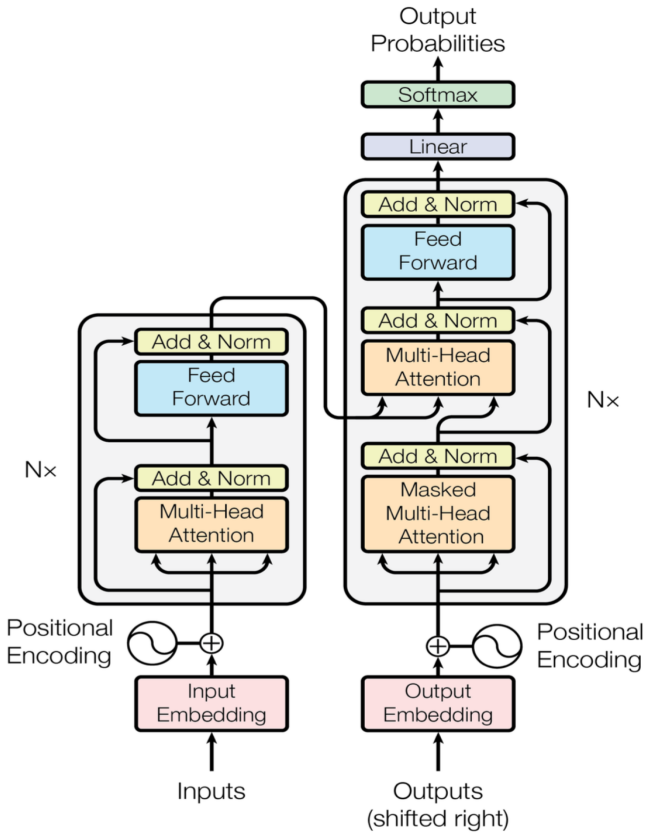


Figure 1: The Transformer - model architecture.

More details:
https://medium.com/inside-machine-learning/what-is-a-transformer-d07dd1fbec04
Papers: https://arxiv.org/abs/1706.03762

Transformers: encoder/ decoder

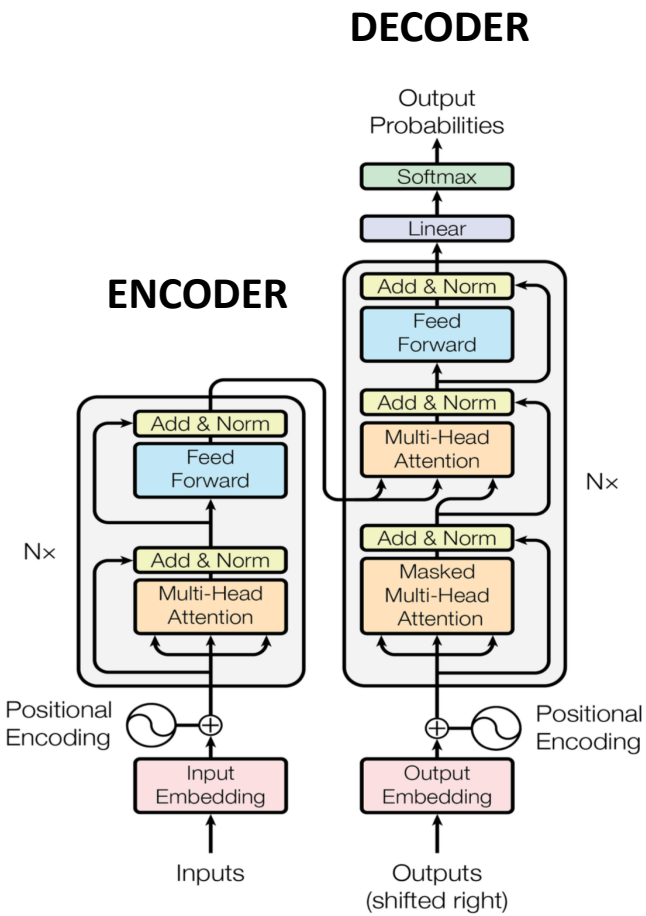
Both encoder and decoder are composed of several modules (Nx in the figure)

Modules are:

- **Multi-Head Attention** – the building blocks that bring innovation to the Transformers
- **Feed-forward** (dense) layers
- **Embeddings** (for inputs and outputs)

Positional encoding – "replaces" the position of the word (token) in the sequence; added to the word embeddings as values in interval [-1; 1] through cosine functions over the index

Some interesting patterns shared by other topologies, such as residual connections, normalization layers are also added



Positional encoding

Previous models did not take into account the position of the tokens in the sequence (neither the Dense layer nor the self attention which is a set-processing mechanism)

A mechanism needs to be used to consider the information about position of the tokens – **positional encoding**

Idea: to add the token's position to each word embedding

	Word order awareness	Context awareness (cross-words interactions)
Bag-of-unigrams	No	No
Bag-of-bigrams	Very limited	No
RNN	Yes	No
Self-attention	No	Yes
Transformer	Yes	Yes

The original paper added this in the form of a value in the range [-1,1] that varied cyclically depending on the position (using the cosine function)

An alternative is to consider **positional embeddings** that are learned !

Positional encoding in torch

```
class PositionalEncoding (nn.Module):

    def __init__(self, d_model, max_len=100):
        super().__init__()
        pe = torch.zeros(max_len, d_model)
        for pos in range(max_len):
            for i in range(0, d_model, 2):
                pe[pos, i] = math.sin(pos / (10000 ** (i/d_model)))
                pe[pos, i+1] = math.cos(pos / (10000 ** (i/d_model)))
        self.pe = pe.unsqueeze(0)

    def forward(self, x):
        return x + self.pe[:, :x.size(1)]
```

$$PE(pos, 2i) = sin(pos/10000^{2i/d_{model}})$$

Pair indexes

$$PE(pos, 2i+1) = cos(pos/10000^{2i/d_{model}})$$

Odd indexes

Python code:
minimal-transformer.ipynb

Transformer block in torch

```
class TransformerBlock (nn.Module):

    def __init__(self, d_model, num_heads, d_ff=256):
        super().__init__()
        self.attn = MultiHeadAttention(d_model, num_heads)
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.ff = FeedForward(d_model, d_ff)

    def forward(self, x):
        attn_output = self.attn(x)          # Multi-head attention
        x = self.norm1(x + attn_output) # Residual + normalization
        ff_output = self.ff(x)              # Feed-forward network
        x = self.norm2(x + ff_output)    # Residual + normalization
        return x
```

```
class FeedForward (nn.Module):

    def __init__(self, d_model, d_ff):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(d_model, d_ff),
            nn.ReLU(),
            nn.Linear(d_ff, d_model)
        )

    def forward(self, x):
        return self.net(x)
```

Residual connections

Python code:
minimal-transformer.ipynb

Transformer in torch: encoder

```
class Transformer (nn.Module):

    def __init__(self, vocab_size, d_model=64,
                 num_heads=4, d_ff=256, num_layers=2):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, d_model)
        self.pos = PositionalEncoding(d_model)
        self.layers = nn.ModuleList(
            [TransformerBlock(d_model, num_heads, d_ff)
             for _ in range(num_layers)])
        self.fc = nn.Linear(d_model, vocab_size)

    def forward(self, x):
        x = self.embedding(x)
        x = self.pos(x)
        for layer in self.layers: x = layer(x)
        return self.fc(x)
```

Stacked transformer blocks

Python code:
minimal-transformer.ipynb

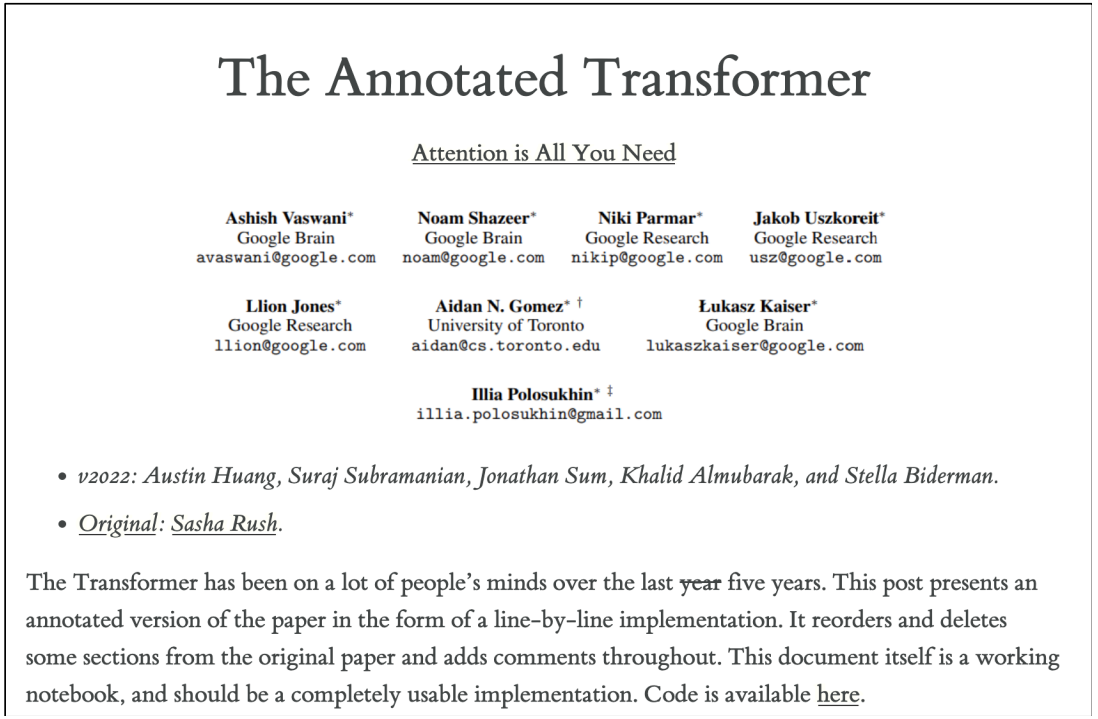
Output shape:
batch_size x
sequence_len x
vocab_size

Transformers: example

A more complete implementation of a transformer in PyTorch is provided in the following links (full code for the Transformers in the original paper and detailed explanation

https://nlp.seas.harvard.edu/annotated-transformer/

https://github.com/harvardnlp/annotated-transformer/ (code repo)



The Annotated Transformer

Attention is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

- *v2022: Austin Huang, Suraj Subramanian, Jonathan Sum, Khalid Almubarak, and Stella Biderman.*
- *Original: Sasha Rush.*

The Transformer has been on a lot of people's minds over the last ~~year~~ five years. This post presents an annotated version of the paper in the form of a line-by-line implementation. It reorders and deletes some sections from the original paper and adds comments throughout. This document itself is a working notebook, and should be a completely usable implementation. Code is available here.

Transformers for text classification

- Transformers (encoders) may be used to support text classification
- Models typically start with embeddings, followed by a transformer encoder, and by one (or more) Dense layers
- Transformers are harder to train than the models seen previously, so many examples may be difficult to run without a GPU

- Next an example for the IMDB dataset similar to the ones in the previous class
- Model uses positional embeddings (learned) + a layer to enable the classification task

Python code:
transformer-imdb.py

Transformers for text classification: IMDB

```
class TransformerClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim, num_heads, ff_dim, max_len):
        super().__init__()

        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        self.pos_embedding = nn.Embedding(max_len, embed_dim)
        self.attention = nn.MultiheadAttention(embed_dim, num_heads, batch_first=True)
        self.norm1 = nn.LayerNorm(embed_dim)
        self.ff = nn.Sequential(
            nn.Linear(embed_dim, ff_dim),
            nn.ReLU(),
            nn.Linear(ff_dim, embed_dim))
        self.norm2 = nn.LayerNorm(embed_dim)
        self.classifier = nn.Linear(embed_dim, 1)
```

Python code:
transformer-imdb.py

Positional embedding

Layer for classification

Transformers - BERT family

- **BERT** (Bidirectional Encoder Representations from Transformers) was created at Google (2018) for word processing and quickly became popular, giving rise to several variants, improving the original and adapting to various types of tasks:
- BERT only uses the **Encoder** part of the transformers, encoding the entire text sequence at once
- It is trained using a token-masking technique in the training data at random positions

Paper: https://arxiv.org/pdf/1810.04805.pdf
https://towardsdatascience.com/bert-explained-state-of-the-art-language-model-for-nlp-f8b21a9b6270
https://www.youtube.com/watch?v=h_U27jBNYI4

Transformers - BERT family – learning task

Masked Language Modeling (Masked LM)

- The objective of this task is to guess the masked tokens.
- Example:

  That's [mask] she [mask] -> That's what she said

Next Sentence Prediction (NSP)

- Given a pair of two sentences, the task is to say whether or not the second follows the first (binary classification).
- Example:

  Input = [CLS] That's [mask] she [mask]. [SEP] Hahaha, nice! [SEP]

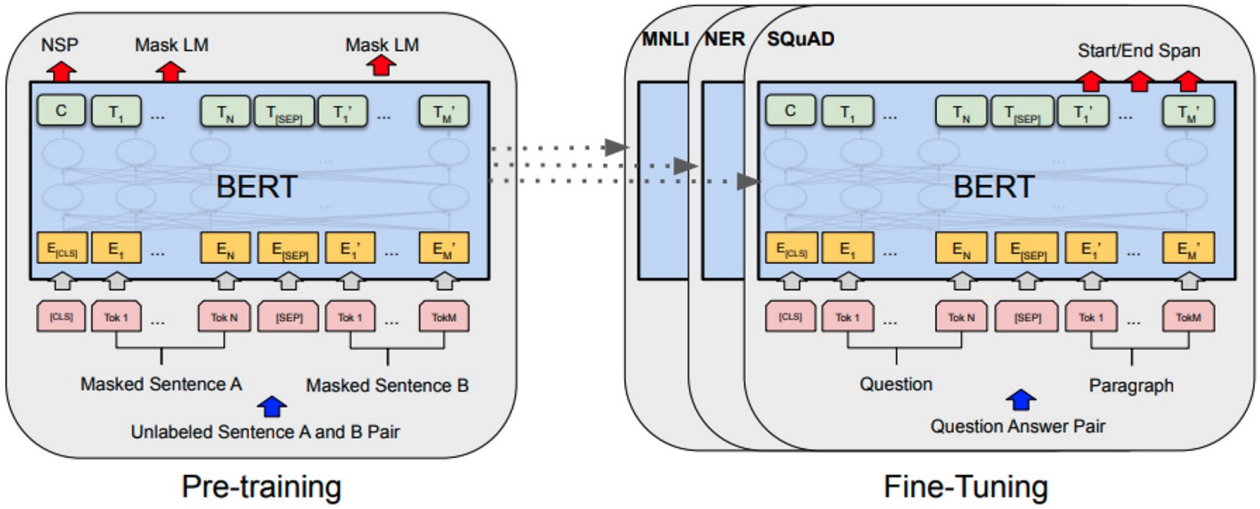
  Label = *IsNext*

  Input = [CLS] That's [mask] she [mask]. [SEP] Dwight, you ignorant [mask]! [SEP]

  Label = *NotNext*

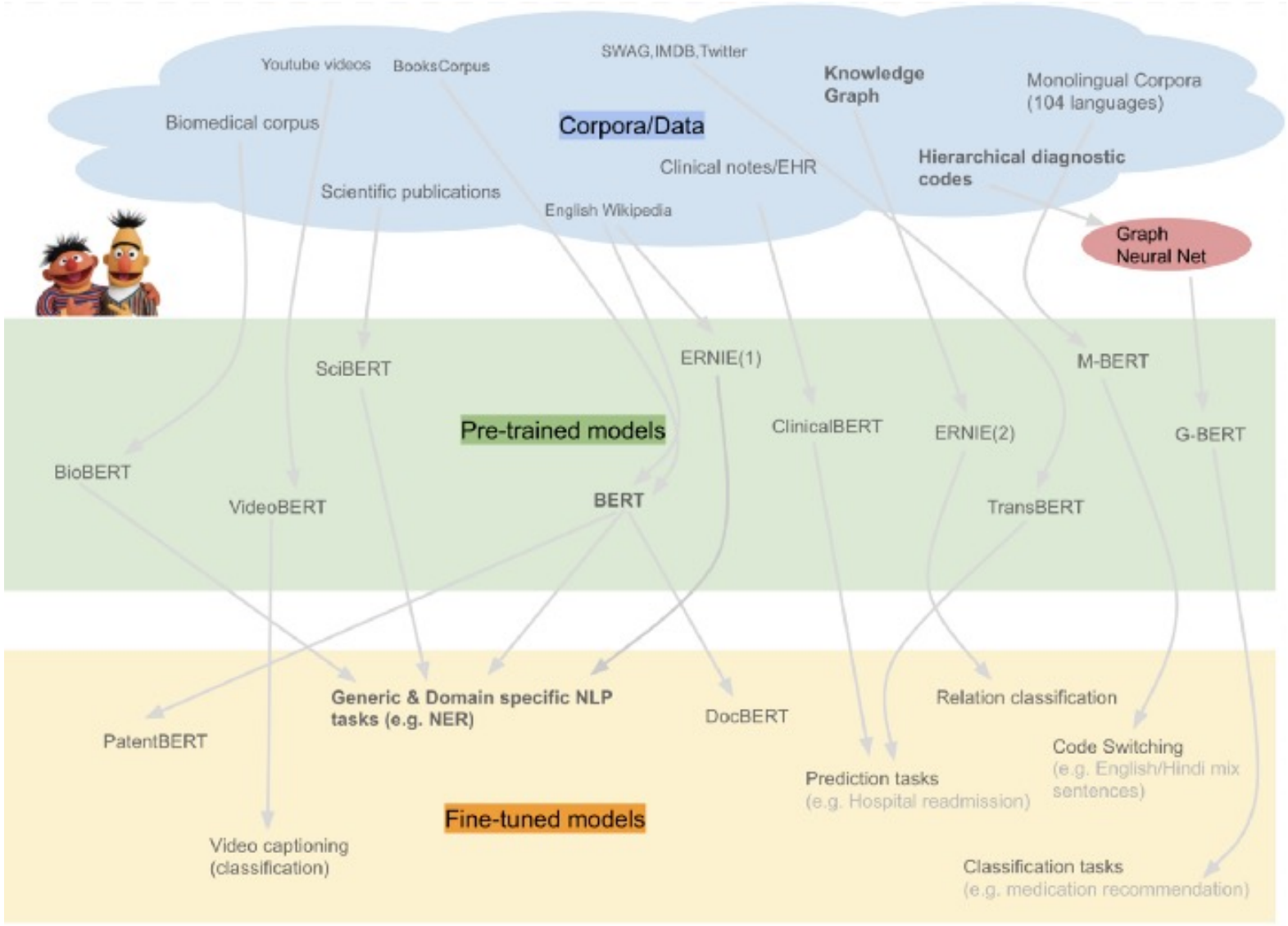
Transformers - BERT family

- **BERT** and several of its successors have been used for numerous text mining applications, using BERT (and variants) to encode the text, and then have a model that uses the BERT output to predict other variables



- Sentiment analysis
- NER
- Question answering
- Document classification
- …

BERT family


Youtube videos
BooksCorpus
SWAG, IMDB, Twitter
Knowledge Graph
Monolingual Corpora (104 languages)
Biomedical corpus
Corpora/Data
Clinical notes/EHR
Hierarchical diagnostic codes
Scientific publications
English Wikipedia
Graph Neural Net
SciBERT
ERNIE(1)
M-BERT
Pre-trained models
ClinicalBERT
ERNIE(2)
G-BERT
BioBERT
VideoBERT
BERT
TransBERT
Generic & Domain specific NLP tasks (e.g. NER)
DocBERT
Relation classification
PatentBERT
Code Switching (e.g. English/Hindi mix sentences)
Prediction tasks (e.g. Hospital readmission)
Fine-tuned models
Video captioning (classification)
Classification tasks (e.g. medication recommendation)

BERT for sentiment analysis (IMDB)

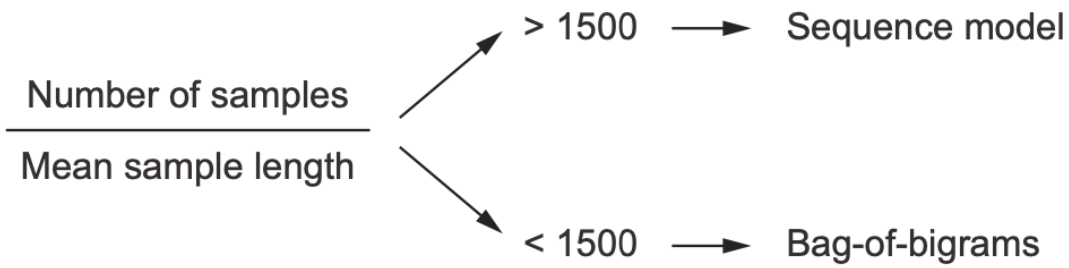
- Example that uses pre-trained BERT for the IMDB example– transfer learning (note: code generated by Claude Sonnet LLM)

- Also fine-tunes the BERT model for 3 epochs in the dataset

Python code:
bert_imdb_sentiment.py

Text classification – which is the best model?

- Although transformers have greatly improved the results on many NLP tasks, more traditional models (e.g. n-grams) may still be the best option in some cases, specially with less data
- Here two factors are relevant: the number of examples (samples) and the mean length (number of words/ tokens)



Rule of thumb from a study conducted by F. Chollet and collaborators

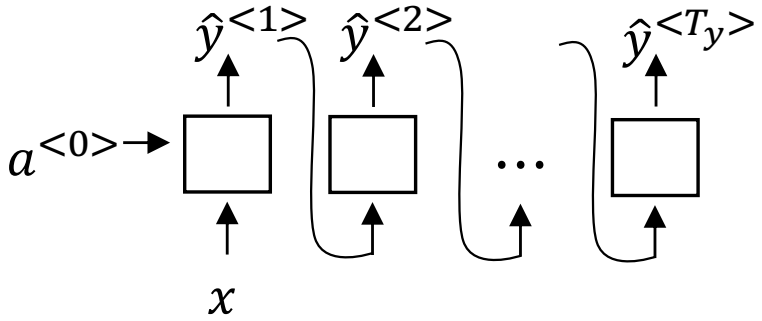
While many other studies have and may be performed, the performance of deep learning models/ transformers is typically more effective when more data are available

What is a Language Model

- **Language model (LM)** - prediction engine that takes a sequence of words and predicts the most likely sequence to come after that sequence. It does this by assigning a probability to likely next sequences and then samples from those to choose one. The process repeats until some stopping criteria is met.
- LMs learn these probabilities by training on large corpora of text.
  - models will be better in some tasks than others.
  - LMs may generate statements that seem plausible, but are actually just random without being grounded in reality.

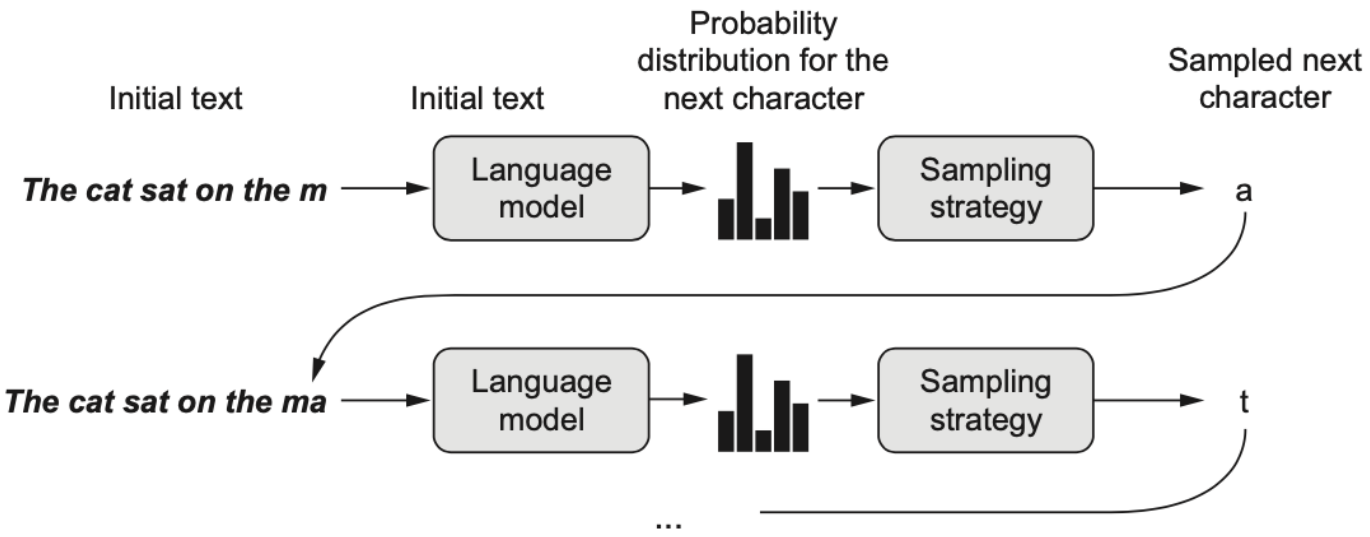
Generating sequences with recurrent NNs

- Recurrent NNs (e.g. RNNs, LSTMs) can be used to generate different types of sequences, through language models, learned from natural language texts
- The idea is to learn probabilities associated to the different symbols of the alphabet (letters, words) from training samples
- When generating, an input sequence is given and the model predicts the next symbol



Generating sequences with LSTMs- example

- Example of text generating letter by letter using LSTMs
- Input – strings of N charaters
- Output – the character N+1 (using a *softmax* layer)



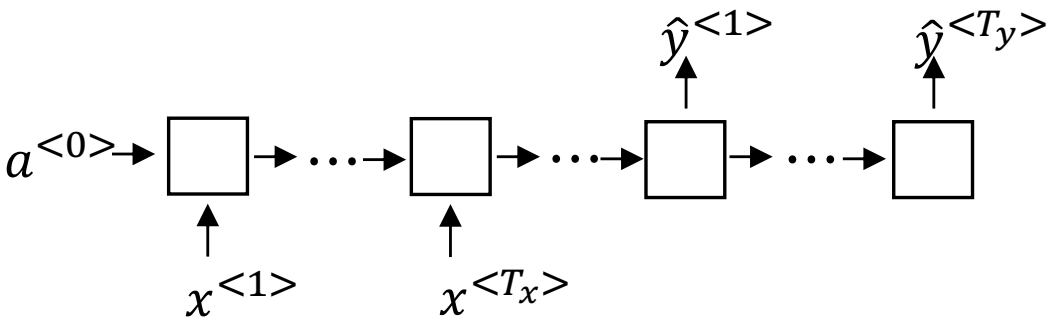
Sequence to sequence models

A **sequence-to-sequence** model takes a sequence as input and translates it into a different sequence

Many different applications:

- Translation
- Text summarization
- Question answering
- Chatbots

Seq to seq models can be implemented by recurrent models (RNNs / LSTMs, etc)



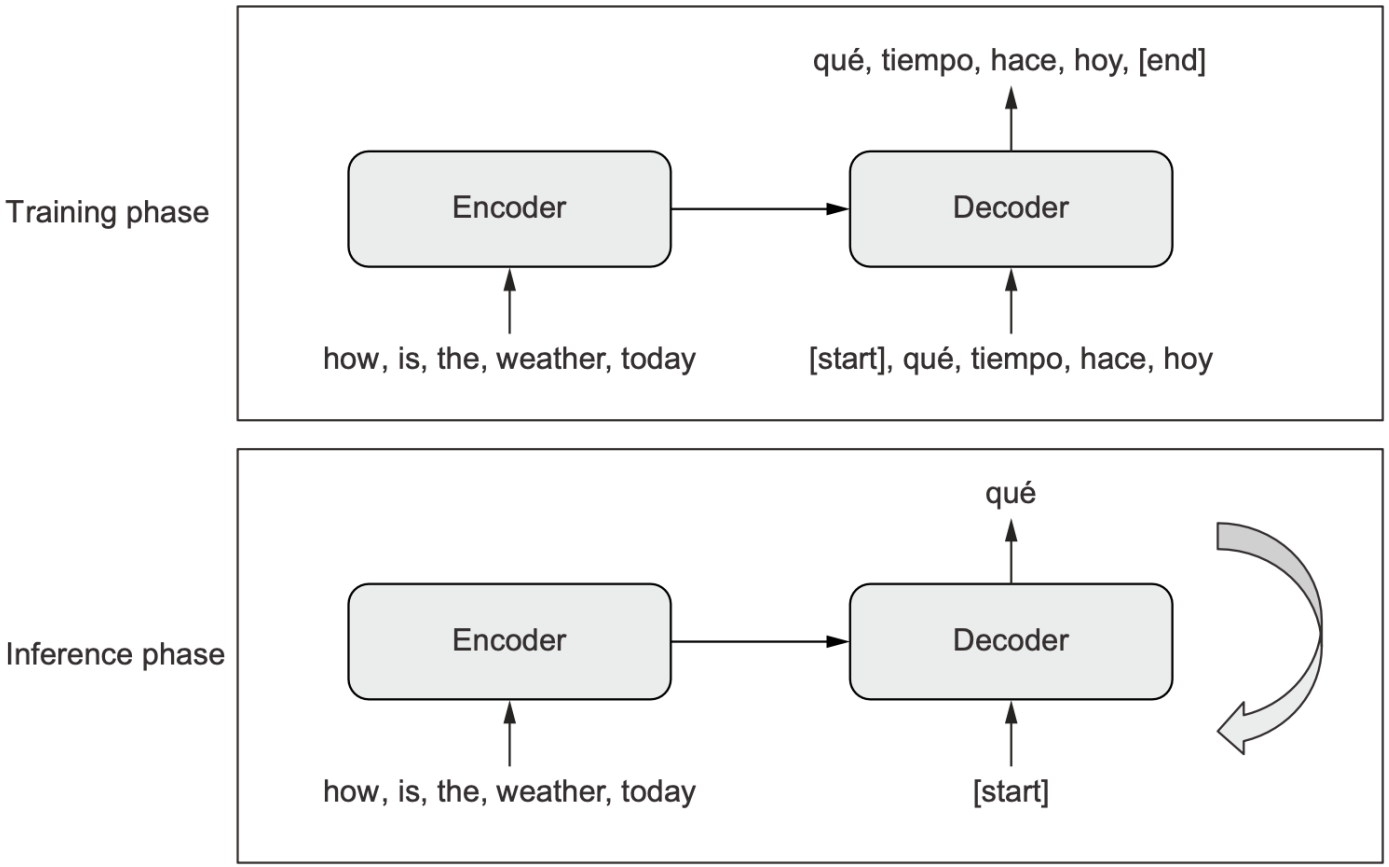
And more recently by transformers

Sequence to sequence models: template

Encoder – decoder architecture

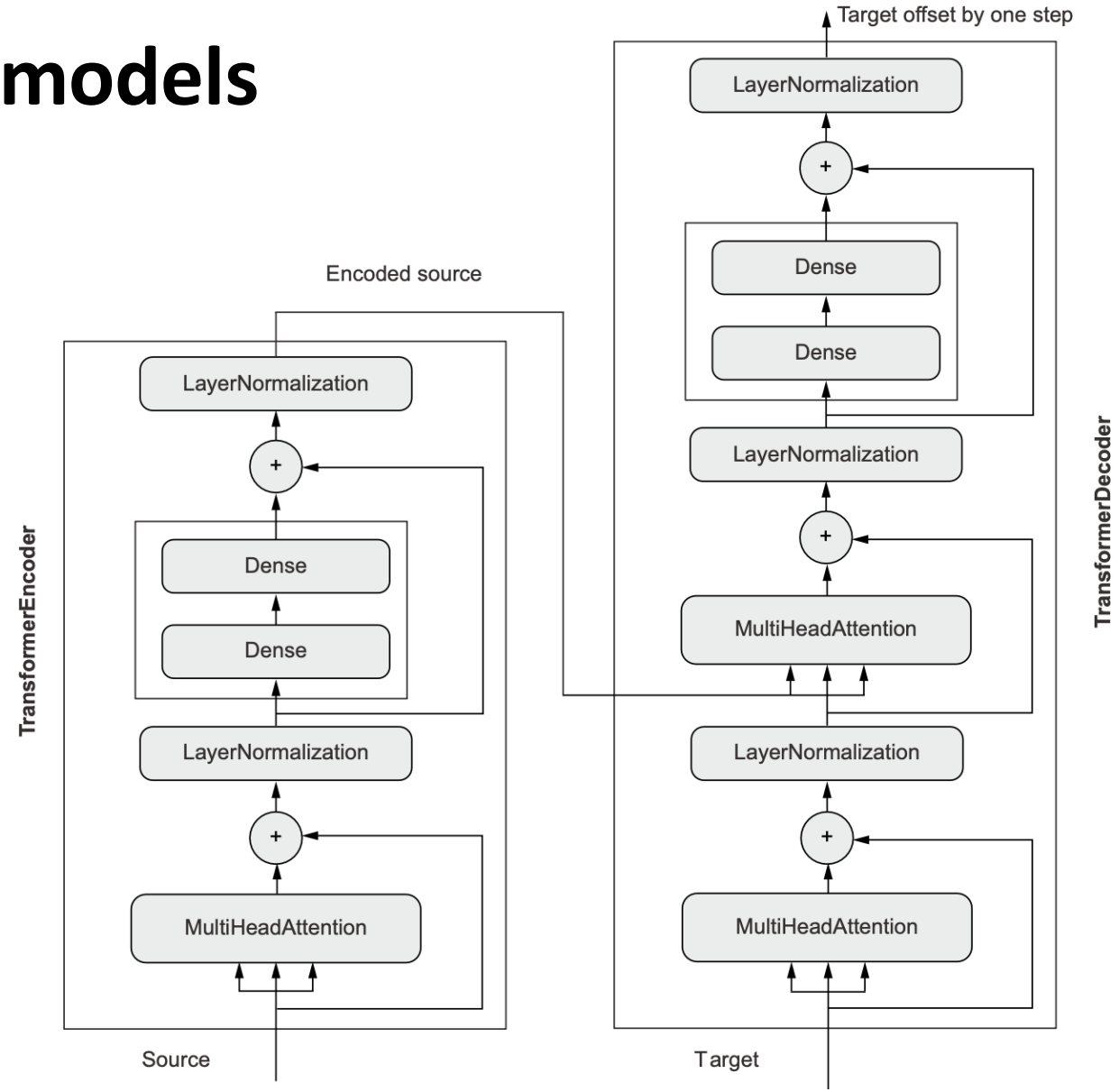
Encoder – converts the input sequence into an intermediate representation (latent space)

Decoder – trained to predict the next token (language model) in the target sequence looking at previous items of the target and the input sequence



Example of language models for translation

- The approach uses a Encoder-Decoder Transformer architecture



Example of language models for translation

- Example of LMs for a simple English → French translation problem:
  - Using a pre-trained encoder-decoder Transformer: MarianMT – Encoder + Decoder
  - Key difference is the use of cross attention, where each decoder token attends to all encoder output states (to generate a French word the decoder looks back at all English encoder states for all tokens)
  - an optional fine-tuning loop on a custom dataset

Python code:
encoder_decoder_translation.py

GPT

- **GPT** (Generative Pretrained Transformer) is an LLM based on the architecture of transformers developed by OpenAI since 2018, being one of BERT's competitors
- It already has several versions of which the last is GPT 5 (2025), with a few trillion parameters
- Only uses the **decoders** part of the transformers – input sequences are fed directly
- It can be used in various word processing and classification applications

Its greatest capacity is the use of the decoder as a **generative model**, generating text from prompts !